21. Rising Temperature

TABLE: WEATHER

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

QUESTION

Write an SQL query to find all dates' Id with higher temperatures compared to its previous dates (yesterday).

Rules:

- Return the result table in any order.

EXPECTED OUTPUT

id
2
4

Reference Solution

```
SELECT w1.id
FROM Weather w1
JOIN Weather w2
  ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

22. Game Play Analysis III

TABLE: ACTIVITY

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
1	3	2017-06-25	1
3	1	2016-03-02	0

QUESTION

Write an SQL query to report for each player and date, how many games played so far by the player. That is, the total number of games played by the player until that date.

Rules:

- Return the result table in any order.

EXPECTED OUTPUT

player_id	event_date	games_played_so_far
1	2016-03-01	5
1	2016-05-02	11
1	2017-06-25	12
3	2016-03-02	0

Reference Solution

```
SELECT
    player_id,
    event_date,
    SUM(games_played) OVER (
        PARTITION BY player_id
        ORDER BY event_date
    ) AS games_played_so_far
FROM Activity;
```

23. Game Play Analysis IV

TABLE: ACTIVITY

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

QUESTION

Write an SQL query to report the fraction of players that logged in again on the day after the day they first logged in, rounded to 2 decimal places.

Rules:

- You need to count the number of players that logged in for at least two consecutive days starting from their first login date, then divide that number by the total number of players.

EXPECTED OUTPUT

fraction
0.33

Reference Solution

```
WITH FirstLogin AS (
    SELECT player_id, MIN(event_date) AS first_login
    FROM Activity
    GROUP BY player_id
)
SELECT ROUND(
    SUM(CASE WHEN a.event_date = DATE_ADD(f.first_login, INTERVAL 1
DAY)
             THEN 1 ELSE 0 END)
    / COUNT(DISTINCT f.player_id), 2
) AS fraction
FROM FirstLogin f
LEFT JOIN Activity a
  ON f.player_id = a.player_id
 AND a.event_date = DATE_ADD(f.first_login, INTERVAL 1 DAY);
```

24. Find Cumulative Salary of an Employee

TABLE: EMPLOYEE

Id	Month	Salary
1	1	20
2	1	20
1	2	30
2	2	30
3	2	40
1	3	40
3	3	60
1	4	60
3	4	70

QUESTION

Write an SQL query to get the cumulative sum of an employee's salary over a period of 3 months but exclude the most recent month.

Rules:

- The result should be displayed by 'Id' ascending, and then by 'Month' descending.

EXPECTED OUTPUT

Id	Month	Salary
1	3	90
1	2	50
1	1	20
2	1	20
3	3	100
3	2	40

Reference Solution

```
SELECT
    Id,
    Month,
    SUM(Salary) OVER(
        PARTITION BY Id
        ORDER BY Month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Salary
FROM Employee
WHERE (Id, Month) NOT IN (
    SELECT Id, MAX(Month)
    FROM Employee
    GROUP BY Id
)
ORDER BY Id ASC, Month DESC;
```

25. Running Total for Different Genders

TABLE: SCORES

player_name	gender	day	score_points
Aron	F	2020-01-01	17
Alice	F	2020-01-07	23
Bajrang	M	2020-01-07	7
Khali	M	2019-12-25	11

QUESTION

Write an SQL query to find the total score for each gender on each day.

Rules:

- Return the result table ordered by gender and day.

EXPECTED OUTPUT

gender	day	total
F	2020-01-01	17
F	2020-01-07	40
M	2019-12-25	11
M	2020-01-07	18

Reference Solution

```
SELECT
    gender,
    day,
    SUM(score_points) OVER(
        PARTITION BY gender
        ORDER BY day
    ) AS total
FROM Scores
ORDER BY gender, day;
```

26. Get the Second Most Recent Activity

TABLE: USERACTIVITY

username	activity	startDate	endDate
Alice	Travel	2020-02-12	2020-02-20
Alice	Dancing	2020-02-21	2020-02-23
Alice	Travel	2020-02-24	2020-02-28
Bob	Travel	2020-02-11	2020-02-18

QUESTION

Write an SQL query to show the second most recent activity of each user.

Rules:

- If the user only has one activity, return that one.
- A user cannot perform more than one activity at the same time.

EXPECTED OUTPUT

username	activity	startDate	endDate
Alice	Dancing	2020-02-21	2020-02-23
Bob	Travel	2020-02-11	2020-02-18

Reference Solution

```
WITH Ranked AS (
    SELECT
        username, activity, startDate, endDate,
        ROW_NUMBER() OVER(PARTITION BY username ORDER BY startDate
DESC) as rn,
        COUNT(*) OVER(PARTITION BY username) as cnt
    FROM UserActivity
)
SELECT username, activity, startDate, endDate
FROM Ranked
WHERE rn = 2 OR cnt = 1;
```

27. The Most Recent Three Orders

TABLE: CUSTOMERS

customer_id	name
1	Winston
2	Jonathan
3	Annabelle
4	Marwan

TABLE: ORDERS

order_id	order_date	customer_id
1	2020-07-31	1
2	2020-07-30	2
3	2020-08-29	3
4	2020-07-29	4
5	2020-06-10	1
6	2020-08-01	2
7	2020-08-01	3

QUESTION

Write an SQL query to find the most recent three orders of each user.

Rules:

- If a user has less than three orders, return all of them.
- Return the result ordered by customer_name ascending, and then by order_date descending.

EXPECTED OUTPUT

name	customer_id	order_id	order_date
Annabelle	3	3	2020-08-29
Annabelle	3	7	2020-08-01
Jonathan	2	6	2020-08-01
Jonathan	2	2	2020-07-30
Marwan	4	4	2020-07-29
Winston	1	1	2020-07-31
Winston	1	5	2020-06-10

Reference Solution

```
WITH RankedOrders AS (
    SELECT
        c.name,
        c.customer_id,
        o.order_id,
        o.order_date,
        ROW_NUMBER() OVER(
            PARTITION BY c.customer_id
            ORDER BY o.order_date DESC
        ) as rn
    FROM Customers c
    JOIN Orders o ON c.customer_id = o.customer_id
)
SELECT name, customer_id, order_id, order_date
FROM RankedOrders
WHERE rn <= 3
ORDER BY name ASC, customer_id ASC, order_date DESC;
```

28. The Most Recent Orders for Each Product

TABLE: PRODUCTS

product_id	product_name
1	keyboard
2	mouse
3	screen

TABLE: ORDERS

order_id	order_date	product_id
1	2020-08-01	1
2	2020-08-02	2
3	2020-08-03	3
4	2020-08-16	1
5	2020-03-01	1

QUESTION

Write an SQL query to find the most recent order(s) for each product.

Rules:

- Return the result table ordered by product_name ascending and product_id ascending.
- If multiple orders have the same most recent date, return all of them.

EXPECTED OUTPUT

product_name	product_id	order_id	order_date
keyboard	1	4	2020-08-16
mouse	2	2	2020-08-02
screen	3	3	2020-08-03

Reference Solution

```
WITH Ranked AS (
    SELECT
        p.product_name,
        p.product_id,
        o.order_id,
        o.order_date,
        RANK() OVER(
            PARTITION BY p.product_id
            ORDER BY o.order_date DESC
        ) as rnk
    FROM Products p
    JOIN Orders o ON p.product_id = o.product_id
)
SELECT product_name, product_id, order_id, order_date
FROM Ranked
WHERE rnk = 1
ORDER BY product_name, product_id;
```

29. Highest Grade For Each Student

TABLE: ENROLLMENTS

student_id	course_id	grade
2	2	95
2	3	95
1	1	90
1	2	99
3	1	80
3	2	75
3	3	82

QUESTION

Write a SQL query to find the highest grade with its corresponding course for each student.

Rules:

- In case of a tie, you should find the course with the smallest course_id.
- Return the result table ordered by student_id in ascending order.

EXPECTED OUTPUT

student_id	course_id	grade
1	2	99
2	2	95
3	3	82

Reference Solution

```
WITH Ranked AS (
    SELECT
        student_id,
        course_id,
        grade,
        ROW_NUMBER() OVER(
            PARTITION BY student_id
            ORDER BY grade DESC, course_id ASC
        ) as rn
    FROM Enrollments
)
SELECT student_id, course_id, grade
FROM Ranked
WHERE rn = 1
ORDER BY student_id;
```

30. Immediate Food Delivery I

TABLE: DELIVERY

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	5	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-11
4	3	2019-08-24	2019-08-26
5	4	2019-08-21	2019-08-22

QUESTION

If the customer's preferred delivery date is the same as the order date, then the order is called immediate. Write an SQL query to find the percentage of immediate orders in the table.

Rules:

- Round to 2 decimal places.

EXPECTED OUTPUT

immediate_percentage
40.0

Reference Solution

```
SELECT ROUND(
    SUM(CASE WHEN order_date = customer_pref_delivery_date
             THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
    2) AS immediate_percentage
FROM Delivery;
```

31. Immediate Food Delivery II

TABLE: DELIVERY

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	2	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-12
4	3	2019-08-24	2019-08-24
5	3	2019-08-21	2019-08-22

QUESTION

Write an SQL query to find the percentage of immediate orders in the first orders of all customers.

Rules:

- First order is the order with the earliest order date for the customer.
- Round to 2 decimal places.

EXPECTED OUTPUT

immediate_percentage
50.0

Reference Solution

```
WITH FirstOrders AS (
    SELECT customer_id, MIN(order_date) as first_order_date
    FROM Delivery
    GROUP BY customer_id
)
SELECT ROUND(
    SUM(CASE WHEN d.order_date = d.customer_pref_delivery_date
             THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
    2) AS immediate_percentage
FROM Delivery d
JOIN FirstOrders f
  ON d.customer_id = f.customer_id
 AND d.order_date = f.first_order_date;
```

32. Restaurant Growth

TABLE: CUSTOMER

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140
7	Anna	2019-01-07	150

QUESTION

Write an SQL query to compute the moving average of how much the customer paid in a seven days window.

Rules:

- Return result table ordered by visited_on in ascending order.
- Average amount should be rounded to two decimal places.

EXPECTED OUTPUT

visited_on	amount	average_amount
2019-01-07	860	122.86

Reference Solution

```
WITH DailyTotal AS (
    SELECT visited_on, SUM(amount) as daily_amount
    FROM Customer
    GROUP BY visited_on
),
MovingAvg AS (
    SELECT
        visited_on,
        SUM(daily_amount) OVER(
            ORDER BY visited_on
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ) as amount,
        ROUND(AVG(daily_amount) OVER(
            ORDER BY visited_on
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ), 2) as average_amount,
        ROW_NUMBER() OVER(ORDER BY visited_on) as rn
    FROM DailyTotal
)
SELECT visited_on, amount, average_amount
FROM MovingAvg
WHERE rn >= 7;
```

33. Number of Calls Between Two Persons

TABLE: CALLS

from_id	to_id	duration
1	2	59
2	1	11
1	3	20
3	4	100
3	4	200
3	4	200
4	3	499

QUESTION

Write an SQL query to report the number of calls and the total call duration between each pair of distinct persons.

Rules:

- person1 < person2 for each pair.
- Return the result table in any order.

EXPECTED OUTPUT

person1	person2	call_count	total_duration
1	2	2	70
1	3	1	20
3	4	4	999

Reference Solution

```
SELECT
    LEAST(from_id, to_id) AS person1,
    GREATEST(from_id, to_id) AS person2,
    COUNT(*) AS call_count,
    SUM(duration) AS total_duration
FROM Calls
GROUP BY
    LEAST(from_id, to_id),
    GREATEST(from_id, to_id);
```

34. Biggest Window Between Visits

TABLE: USERVISITS

user_id	visit_date
1	2020-11-28
1	2020-10-20
1	2020-12-3
2	2020-10-5
2	2020-12-9
3	2020-11-11

QUESTION

Assume today's date is '2021-1-1'. Write an SQL query that returns the biggest window of days between two visits for each user.

Rules:

- Biggest window is the max difference in days between consecutive visits, or from last visit to '2021-1-1'.
- Return the result table ordered by user_id.

EXPECTED OUTPUT

user_id	biggest_window
1	39
2	65
3	51

Reference Solution

```
WITH NextVisits AS (
    SELECT
        user_id,
        visit_date,
        LEAD(visit_date, 1, '2021-01-01') OVER(
            PARTITION BY user_id
            ORDER BY visit_date
        ) as next_visit
    FROM UserVisits
)
SELECT
    user_id,
    MAX(DATEDIFF(next_visit, visit_date)) AS biggest_window
FROM NextVisits
GROUP BY user_id
ORDER BY user_id;
```

35. Find Total Time Spent by Each Employee

TABLE: EMPLOYEES

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-03	1	42
2	2020-11-28	3	33
2	2020-12-09	47	74

QUESTION

Write an SQL query to calculate the total time in minutes spent by each employee on each day at the office.

Rules:

- Note that within one day, an employee can enter and leave more than once. The time spent is out_time - in_time.

EXPECTED OUTPUT

day	emp_id	total_time
2020-11-28	1	173
2020-11-28	2	30
2020-12-03	1	41
2020-12-09	2	27

Reference Solution

```
SELECT
    event_day AS day,
    emp_id,
    SUM(out_time - in_time) AS total_time
FROM Employees
GROUP BY event_day, emp_id;
```

36. Percentage of Users Attended Contest

TABLE: USERS

user_id	user_name
6	Alice
2	Bob
7	Alex

TABLE: REGISTER

contest_id	user_id
215	6
209	2
208	2
210	6
208	6
209	7
209	6
215	7
208	7
210	2
207	2
210	7

QUESTION

Write an SQL query to find the percentage of the users registered in each contest rounded to two decimals.

Rules:

- Return the result table ordered by percentage in descending order. In case of a tie, order it by contest_id in ascending order.

EXPECTED OUTPUT

contest_id	percentage
208	100.0
209	100.0
210	100.0
215	66.67

Reference Solution

```
SELECT
    contest_id,
    ROUND(COUNT(user_id) * 100.0 / (SELECT COUNT(*) FROM Users), 2) AS
percentage
FROM Register
GROUP BY contest_id
ORDER BY percentage DESC, contest_id ASC;
```

207	33.33

37. Confirmation Rate

TABLE: SIGNUPS

user_id	time_stamp
3	2020-03-21
7	2020-01-04
2	2020-07-29
6	2020-12-09

TABLE: CONFIRMATIONS

user_id	time_stamp	action
3	2021-01-06	timeout
3	2021-07-14	timeout
7	2021-06-12	confirmed
7	2021-06-13	confirmed
7	2021-06-14	confirmed
2	2021-01-22	confirmed
2	2021-02-28	timeout

QUESTION

The confirmation rate of a user is the number of 'confirmed' messages divided by the total number of requested confirmation messages. Write a query to find the confirmation rate of each user.

Rules:

- If a user did not request any confirmation messages, their confirmation rate is 0. Round to 2 decimal places.

EXPECTED OUTPUT

user_id	confirmation_rate
6	0.0
3	0.0
7	1.0
2	0.5

Reference Solution

```
SELECT
    s.user_id,
    ROUND(
        AVG(CASE WHEN c.action = 'confirmed' THEN 1.0 ELSE 0.0 END),
    2) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c
  ON s.user_id = c.user_id
GROUP BY s.user_id;
```

38. League Statistics

TABLE: TEAMS

team_id	team_name
1	Ajax
4	Dortmund
6	Arsenal

TABLE: MATCHES

home_team_id	away_team_id	home_team_goals	away_team_goals
1	4	0	1
1	6	3	3
4	1	5	2
6	1	0	0

QUESTION

Write an SQL query to report the statistics of the league. Statistics should include team_name, matches_played, points, goal_for, goal_against, goal_diff.

Rules:

- Points: 3 for win, 1 for draw, 0 for loss.
- Order by points DESC, goal_diff DESC, team_name ASC.

EXPECTED OUTPUT

team_name	matches_played	points	goal_for	goal_against	goal_diff
Dortmund	2	6	6	2	4
Arsenal	2	2	3	3	0
Ajax	4	2	5	9	-4

Reference Solution

```
WITH AllMatches AS (
    SELECT
        home_team_id AS team_id,
        home_team_goals AS goals_for,
        away_team_goals AS goals_against
    FROM Matches
    UNION ALL
    SELECT
        away_team_id AS team_id,
        away_team_goals AS goals_for,
        home_team_goals AS goals_against
    FROM Matches
)
SELECT
    t.team_name,
    COUNT(*) AS matches_played,
    SUM(CASE
        WHEN goals_for > goals_against THEN 3
        WHEN goals_for = goals_against THEN 1
        ELSE 0 END) AS points,
    SUM(goals_for) AS goal_for,
    SUM(goals_against) AS goal_against,
    SUM(goals_for - goals_against) AS goal_diff
FROM AllMatches m
JOIN Teams t ON m.team_id = t.team_id
GROUP BY t.team_name
ORDER BY points DESC, goal_diff DESC, t.team_name ASC;
```

39. Group Sold Products By The Date

TABLE: ACTIVITIES

sell_date	product
2020-05-30	Headphone
2020-06-01	Pencil
2020-06-02	Mask
2020-05-30	Basketball
2020-06-01	Bible
2020-06-02	Mask
2020-05-30	T-Shirt

QUESTION

Write an SQL query to find for each date the number of different products sold and their names.

Rules:

- The sold products names for each date should be sorted lexicographically.
- Return the result table ordered by sell_date.

EXPECTED OUTPUT

sell_date	num_sold	products
2020-05-30	3	Basketball,Headphone,T-Shirt
2020-06-01	2	Bible,Pencil
2020-06-02	1	Mask

Reference Solution

```
SELECT
    sell_date,
    COUNT(DISTINCT product) AS num_sold,
    GROUP_CONCAT(DISTINCT product ORDER BY product ASC SEPARATOR ',')
AS products
FROM Activities
GROUP BY sell_date
ORDER BY sell_date;
```

40. The Number of Employees Which Report to Each Employee

TABLE: EMPLOYEES

employee_id	name	reports_to	age
9	Hercy	Null	43
6	Alice	9	41
4	Bob	9	36
2	Winston	Null	37

QUESTION

Write an SQL query to report the ids and the names of all managers, the number of employees who report directly to them, and the average age of the reports rounded to the nearest integer.

Rules:

- Return the result table ordered by employee_id.

EXPECTED OUTPUT

employee_id	name	reports_count	average_age
9	Hercy	2	39

Reference Solution

```
SELECT
    mgr.employee_id,
    mgr.name,
    COUNT(emp.employee_id) AS reports_count,
    ROUND(AVG(emp.age), 0) AS average_age
FROM Employees mgr
JOIN Employees emp
  ON mgr.employee_id = emp.reports_to
GROUP BY mgr.employee_id, mgr.name
ORDER BY mgr.employee_id;
```